

Technical Requirements Document (TRD): Share Sathi

1. Executive Summary

Technical Vision: Share Sathi will be built as a serverless Jamstack application leveraging Next.js 14+ with App Router for optimal performance and developer experience. The architecture follows a microservices-like pattern with serverless functions handling specific tasks (data fetching, processing, posting) while maintaining a simple, cost-effective infrastructure.

Key Technical Decisions:

- Next.js 14 + App Router: Modern React framework with excellent Vercel integration
- Vercel Platform: Zero-config deployment, built-in cron jobs, and serverless functions
- Turso (libSQL): Edge-ready SQLite database for simple data storage needs
- Serverless Architecture: Cost-effective, scalable, and maintenance-free
- Multiple Data Source Fallback: Resilience against API failures

Risk Assessment:

- High: Facebook API changes/rate limiting - Mitigation: Version pinning, queue system
- Medium: NEPSE data source reliability - Mitigation: Multiple fallback sources
- Low: Vercel platform issues - Mitigation: Simple architecture allows easy migration

2. Architecture Overview

High-Level System Architecture:

Data Sources Vercel Edge Facebook
(NEPSE APIs) Serverless Graph API
Turso DB
Admin Dashboard
(Next.js App)

Component Description:

- Data Fetcher Service: Serverless function that calls NEPSE APIs in priority order
- Content Processor: Transforms raw data into formatted Facebook content
- Facebook Poster: Handles authentication and posting via Graph API
- Turso Database: Stores historical data, post records, and system logs
- Admin Dashboard: Next.js frontend for monitoring and management

Data Flow:

- 3:05 PM NPT: Vercel cron triggers data fetcher
- Priority-based fetching: Official NEPSE Merolagani Sharesansar
- Data validation: Schema validation and completeness check
- Content formatting: Create engaging Facebook post with emojis
- Facebook posting: Authenticate and post via Graph API
- Logging: Record success/failure in Turso database
- Notification: Send alerts on failures via WhatsApp/email

3. Technology Stack

Layer	Technology	Version	Justification
Frontend	Next.js	14.0+	App Router, SSR, Vercel optimization
	React	18.0+	Modern hooks, concurrent features
	Tailwind CSS	3.0+	Utility-first, responsive design
	TypeScript	5.0+	Type safety, better developer experience
Backend	Next.js API Routes	14.0+	Unified full-stack framework
Serverless Functions	-	-	Vercel-native, auto-scaling
	Node.js	18+ LTS	Stable, long-term support
Database	Turso (libSQL)	Latest	Edge SQLite, simple, affordable
	Drizzle ORM	0.30+	Type-safe, Turso-optimized
Infrastructure	Vercel Platform	-	Zero-config, built-in cron, serverless
	Vercel Cron Jobs	-	Scheduled tasks for automation
	GitHub Actions	-	CI/CD pipeline

Third-party | Facebook Graph API | v19.0 | Official Facebook integration
Resend | Latest | Transactional email notifications
Twilio WhatsApp API | - | WhatsApp business notifications
Axios | 1.0+ | HTTP client for API calls

4. Frontend Requirements

Component Architecture:

// App structure

src/

app/

dashboard/

page.tsx # Main dashboard

layout.tsx # Dashboard layout

api/

cron/

route.ts # Cron job endpoint

facebook/

route.ts # Facebook API proxy

globals.css # Tailwind imports

layout.tsx # Root layout

State Management: React hooks (useState, useContext) for simple global state

Authentication Context: Admin auth state

System Status Context: Real-time monitoring data

No Redux/Zustand: Minimal state needs don't justify complex management

Routing Structure:

`/` Dashboard homepage

`/api/cron/nepse-fetch` Data fetching endpoint

`/api/facebook/post` Facebook posting endpoint

Protected routes with middleware authentication

API Integration Patterns:

// Example API call pattern

```
const fetchNepseData = async (source: string) => {
```

```
  try {
```

```
    const response = await fetch(`/api/nepse?source=${source}`);
```

```
    if (!response.ok) throw new Error('Fetch failed');
```

```
    return await response.json();
```

```
  } catch (error) {
```

```
    // Fallback to next source
```

```
    return null;
```

```
  }
```

```
};
```

Responsive Design Breakpoints:

Mobile: < 768px (Tailwind: `sm:`)

Tablet: 768px - 1024px (`md:`)

Desktop: 1024px+ (`lg:`)

Performance Budgets:

LCP: < 2.5s

FID: < 100ms

CLS: < 0.1

JS Bundle: < 150kB

CSS: < 50kB

5. Backend Requirements

API Design: RESTful endpoints with consistent error handling

// API Response Standard

```
interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: string;
  timestamp: string;
}
```

Authentication & Authorization:

Admin Auth: Simple password protection (no multi-user needed)

API Keys: Environment variables for third-party services

Facebook Auth: Long-lived page access token with `pages\_manage\_posts` permission

Error Handling Strategy:

// Unified error handler

```
const handleError = (error: unknown, context: string) => {
  const errorMessage = error instanceof Error ? error.message : 'Unknown error';
  logError(context, errorMessage);
  sendNotification(`${context} failed: ${errorMessage}`);
  return { success: false, error: errorMessage };
};
```

Rate Limiting & Throttling:

Facebook API: 200 calls/hour limit implement queue system

NEPSE APIs: Conservative 1 request/5 seconds to avoid blocks

Retry Logic: Exponential backoff for temporary failures

Background Job Processing:

Vercel Cron Jobs: Scheduled execution at 3:05 PM NPT daily

Queue System: In-memory queue for Facebook posting retries

State Management: Turso database tracks job status

File Upload Handling: Not required for v1.0

6. Database Design

Schema Design Principles:

Simple relational structure: Few tables with clear relationships

Audit logging: All actions timestamped and recorded

Data integrity: Foreign keys and constraints where appropriate

Minimal indexing: Only essential columns indexed

Database Schema:

-- Main tables

```
CREATE TABLE market_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  date TEXT NOT NULL,
  source TEXT NOT NULL,
  index_value REAL,
  turnover REAL,
  volume INTEGER,
  raw_data TEXT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE facebook_posts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  market_data_id INTEGER REFERENCES market_data(id),
  post_id TEXT, -- Facebook post ID
  content TEXT NOT NULL,
  status TEXT CHECK(status IN ('pending', 'success', 'failed')),
  error_message TEXT,
  posted_at DATETIME,
```

```
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE system_logs (
id INTEGER PRIMARY KEY AUTOINCREMENT,
level TEXT CHECK(level IN ('info', 'warning', 'error')),
message TEXT NOT NULL,
context TEXT,
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Indexing Strategy:

- `market\_data(date)` Quick date-based queries
- `facebook\_posts(status)` Filter by post status
- `system\_logs(timestamp)` Time-based log analysis

Data Migration Approach: Drizzle ORM migrations

- Version-controlled schema changes
- Zero-downtime deployments
- Rollback capability

Backup Procedures:

- Turso Automated Backups: Daily automatic backups
- Export Script: Weekly JSON export to cloud storage
- Retention: 30 days of automated backups, 1 year of weekly exports

7. API Specification

Data Endpoints

Method	Endpoint	Description	Auth	Request Body	Response
GET	/api/data/nepse	Fetch NEPSE data	None	{source: string}	MarketData
POST	/api/data/process	Process raw data	Admin	RawData	FormattedContent

Facebook Endpoints

Method	Endpoint	Description	Auth	Request Body	Response
POST	/api/facebook/post	Post to Facebook	API Key	{content: string}	{postId: string}
GET	/api/facebook/status	Check post status	Admin	{postId: string}	PostStatus

System Endpoints

Method	Endpoint	Description	Auth	Request Body	Response
GET	/api/system/status	System health	None	None	SystemStatus
POST	/api/system/notify	Send notification	API Key	NotificationPayload	{success: boolean}

Error Response Format:

```
{
  "error": {
    "code": "FACEBOOK_RATE_LIMIT",
    "message": "Rate limit exceeded",
    "details": "Retry after 3600 seconds",
    "timestamp": "2024-06-05T10:30:00Z"
  }
}
```

8. Security Requirements

Authentication Flow:

Admin Login Password Verification JWT Token API Authentication

Authorization Model: Simple RBAC

- Admin: Full access to all endpoints and dashboard
- System: API key access to specific endpoints
- Public: Read-only access to status endpoints

Data Encryption:

- At Rest: Turso database encryption
- In Transit: TLS 1.3 for all communications

Secrets: Vercel Environment Variables encryption
Input Validation & Sanitization:
// Zod schema validation
const postSchema = z.object({
 content: z.string().min(10).max(2000),
 scheduledFor: z.date().optional()
});
const safeData = postSchema.parse(requestBody);
CSRF/XSS Protection:
 CSRF: Next.js built-in protection
 XSS: Content sanitization for user-generated content (future)
 CORS: Restricted to trusted domains
Secrets Management:
 Vercel Env Variables: All secrets stored encrypted
 No Hardcoded Secrets: Codebase contains zero secrets
 Rotation Policy: Quarterly secret rotation

9. Performance Requirements

Response Time Targets:
 p50: < 200ms for all API responses
 p95: < 500ms for data processing
 p99: < 1000ms for Facebook API calls
Throughput Targets:
 Data Fetching: 10 requests/minute during peak
 Facebook Posting: 5 posts/minute (well under limits)
 Dashboard: Support 10 concurrent admin users
Caching Strategy:
 CDN Caching: Static assets cached at edge
 API Cache: 5-minute cache for market data
 Browser Cache: Aggressive caching for static resources
Database Optimization:
 Query Optimization: Indexed queries only
 Connection Pooling: Turso connection management
 Selective Fields: Never `SELECT \*`
Frontend Performance:
 LCP: < 2.0s (Largest Contentful Paint)
 FID: < 50ms (First Input Delay)
 CLS: < 0.05 (Cumulative Layout Shift)
 Bundle Size: < 100kB gzipped

10. Deployment & Infrastructure

Environment Strategy:
 Development: `dev.share-sathi.vercel.app` Preview deployments
 Production: `share-sathi.vercel.app` Production domain
CI/CD Pipeline:

GitHub Actions workflow

```
name: Deploy to Vercel  
on:  
  push:  
    branches: [main]  
  pull_request:  
    branches: [main]  
jobs:  
  deploy:
```

runs-on: ubuntu-latest

steps:

uses: actions/checkout@v4

uses: vercel/action@v4

with:

vercel-token: \${{ secrets.VERCEL_TOKEN }}

vercel-org-id: \${{ secrets.ORG_ID }}

vercel-project-id: \${{ secrets.PROJECT_ID }}

Monitoring & Alerting:

Vercel Analytics: Performance monitoring

Custom Logging: Turso-based error tracking

Uptime Monitoring: 3rd party service (StatusCake)

Error Tracking: Console logging + notifications

Scaling Strategy:

Horizontal Scaling: Vercel automatic scaling

Database Scaling: Turso auto-scaling

CDN: Global edge network for static assets

Disaster Recovery:

Database Backups: Daily automated backups

Environment Recovery: Vercel project cloning

Manual Override: Admin dashboard fallback posting

11. Testing Strategy

Unit Testing:

Framework: Jest + Testing Library

Coverage Target: 80%+ for critical paths

Focus: Utility functions, data processing, API handlers

Integration Testing:

API Testing: Supertest for endpoint testing

Database Testing: Test database with seeded data

Third-party Mocks: Mock service workers for APIs

E2E Testing:

Framework: Playwright

Critical Paths: Data fetch process post flow

Dashboard Testing: Admin functionality testing

Performance Testing:

Lighthouse CI: Integrated with PR checks

API Load Testing: k6 for endpoint performance

Monitoring: Real-user monitoring in production

Security Testing:

Dependency Scanning: Dependabot security updates

Code Scanning: GitHub CodeQL analysis

Manual Review: Quarterly security audit

12. Action Items

****P0 - Critical Path (Week 1)****

Setup Next.js + Turso project with TypeScript configuration

Configure Vercel project with environment variables

Create database schema and Drizzle ORM setup

Implement NEPSE data fetcher with priority fallback logic

Set up Facebook Developer app with page access token

****P1 - Core Functionality (Week 2)****

Build content formatter with emoji support and gainers/losers focus

Implement Facebook poster with Graph API integration

Create cron job system for 3:05 PM NPT automation

Build error notification system with WhatsApp + email

Implement market holiday handling with "Market Closed" posts

****P2 - Admin Dashboard (Week 3)****

- Create basic dashboard with system status monitoring
- Implement post history view with status indicators
- Add manual controls for retry and override functionality
- Create logo and branding for Share Sathi Facebook page
- Set up monitoring and alerting infrastructure

****Open Technical Decisions****

- WhatsApp API provider choice: Twilio vs native Business API
- Email service provider: Resend vs SendGrid vs NodeMailer
- Error tracking service: Custom vs third-party (Sentry)
- Uptime monitoring tool: StatusCake vs UptimeRobot
- Backup storage solution: AWS S3 vs Google Cloud Storage

****Immediate Next Steps****

- Create GitHub repository with initial Next.js setup
- Set up Turso database and configure connection
- Apply for Facebook Developer access and page permissions
- Register for WhatsApp Business API access
- Design Share Sathi logo for Facebook page

Kiran, this TRD provides the complete technical blueprint for Share Sathi. The architecture is simple yet robust, leveraging modern tools that ensure reliability and minimal maintenance. Would you like me to proceed with setting up the initial project structure?

